Divergence: formatting a script can turn it into a different program, across 37+ JavaScript processors

Divergence: formatting a script can turn it into a different program, across 37+ JavaScript processors - Robert Vähhi, TrustSig.

- Published: 2026-10-17
- Original: <https://trustsig.eu/blog/javascript-divergence/>
- Preserved from: https://trustsig.eu/blog/javascript-divergence/ (live) on 2026-10-21
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

The first step to almost any JavaScript reversal is to look at a processed (*formatted/cleaned/transformed*) version of the script, whether that be DevTools in the browser, a formatter (*e.g. prettier, js-beautify*) or a deobfuscator (*e.g. WebCrack, deobfuscate.io, de4js*).

But have you ever given thought that even a simple formatting pass, like the one Chromium DevTools does automatically, could make the script run an entirely different program altogether?

You're probably thinking this needs source maps or a `.toString` check of the code on runtime, but it's quite the opposite. The formatting itself statically converts the script into logic of the script author's choosing. Without needing to abuse any source maps, runtime integrity or code checks or network. It purely works because of bugs in the JavaScript processors.

You can have a script that plays Tetris and as soon as you run it through a processor (*e.g. DevTools, prettier, esbuild*) it becomes Snake. The demo is further down.

It sounds impossible, but it's exactly what this blog will talk about and demo. This also isn't a niche bug that can only be found in one library or tool. The pattern can be found in every single JavaScript processor (even Chromium & Firefox DevTools), and most of them have more than a few. I found **186** unique techniques across 37+ of them, including:

`prettier, js-beautify, babel, escodegen, astring, swc, esbuild, terser, uglify-js, oxc, calmjs.parse, wakaru, javascript-deobfuscator, obfuscator-io-deobfuscator, flast, new-`

`datadome-deobfuscator, deobfuscate-js, kuizuo js-deobfuscator, webcrack, restringer, synchrony`

By JavaScript processor I mean anything that turns JavaScript into other JavaScript: formatters, AST->JS printers, minifiers and deobfuscators.

Divergence

I have started to call this anti-reversing technique "**divergence**". If you're familiar with native software decompiling you might know it as "disassembly desynchronization", but as far as I found it has never been documented for JavaScript.

What has existed for decades is detecting modified JS on runtime, and weird syntax that crashes parsers. The closest thing on the web is mXSS (benign HTML turned into executable JavaScript by browser DOM serialization differentials), which is much less useful for anti-reverse engineering.

Why would you want this

When an LLM or a human reverser is trying to understand obfuscated JavaScript, one of the first things they do is process it to make it more readable. That can be a custom processor (*JS->AST and back to JS*), a common formatter (*e.g. DevTools, prettier, js-beautify*) that's also what formats code in their editor, or a full-blown deobfuscator (*e.g. de4js, webcrack*).

With deobfuscators you might expect the logic to differ a bit. But it's very unlikely you'd suspect a script was built so that just converting it to AST and back to JS results in new, author controlled logic. Logic that still works well enough to run without raising any suspicion, but is the wrong program altogether. So you end up reversing the wrong thing, and you might not even realize it for a long time.

This is important: the diverged script is a decoy, not a block. It's not meant to stop execution or give instant feedback that the processor changed the logic. If the diverged code crashes right away, or the script gives any other sign that the divergence was triggered, any reverser will instantly know the processor changed something.

Against LLM agents

LLM agents reversing JavaScript will **almost always** format it at the very least, since anything large and complex enough can't just be loaded into context or grepped through. They never even throw out a thought about verifying it with anything more than running the script and seeing whether it still works. That makes divergence an insanely powerful defense against LLM reversers.

Against reimplementation

Many anti-cheats, anti-bots and other systems rely heavily on their client side code being protected well enough that it at the very least takes a long time to reverse and

understand. Classically, once you've reversed a script and reimplemented it one-to-one, you can be basically fully confident the logic works the same way.

With a divergence in the script, even one that just swaps out a single number, the reimplementation sends a value the real script never would. That's a signal the server can detect and act on with delayed bans or monitoring.

The Tetris / Snake demo

The demo combines many of the divergence techniques to change the bytecode fed into a small custom VM. The original script contains Tetris. Once it's processed by any of the processing tools, it creates the bytecode for Snake instead and erases Tetris.

This is an extreme example of what's possible, and it works even with DevTools output.

Check out the Tetris / Snake demo here: divergence-game.trustsig.eu

Playground

You can play around with it better in the playground at divergence.trustsig.eu

Before the examples

I'm only covering divergences that produce controlled and working code. Anything that just crashes the JS processor or crashes at runtime is uninteresting, so it's not here.

There is no single technique that works on every processor at once. Each one needs a bit of a different approach, and gives you less or more control over what code can be diverged and how. Techniques in upstream libraries (*e.g. astring, swc, esbuild*) do carry over to every tool using them though.

Most of these tools also have different modes (*minify, format, print*) and config options (*ES targets, ESM/CJS, ECMAScript editions*), so X having a divergence bug does not mean it triggers every time X runs.

Safari DevTools is much less used, so I did not bother researching it.

The six types of divergence

Not all divergences are equal. The type a tool is vulnerable to correlates quite closely with what the tool was built to do. Simple formatters (*e.g. Chromium/Firefox DevTools*) have simple divergences, heavy deobfuscators (*e.g. WebCrack*) have really severe ones.

I define severity by how much code you can control changing. If all you can do is make a `0` become a `1`, that's *LOW* severity. If you can delete an entire section of code, that's *HIGH*.

From the smallest change to the biggest:

| Type | What changes | Example | Severity | | Rerouted | all code kept in the same shape, but a different value or path runs. | Chromium DevTools | LOW | | Rewritten | logic kept

but expressed as another construct (lowered, inlined, reordered). | WebCrack | LOW | | | Folded | an expression or condition replaced by a precomputed, wrong constant. | Restringer | MEDIUM | | | Stripped | side-effecting statements, calls or arguments deleted, the function kept. | new-datadome-deobfuscator | MEDIUM | | | Pruned | a branch or case deleted, the other side kept. | obfuscator-io-deobfuscator | HIGH | | | Erased | a whole function or body emptied or removed. | Terser | HIGH | |

Below is one example of each, picked out of the 186. All of them are in the playground at the end.

Rerouted: Chromium DevTools

The code stays logically almost identical, just a single operation behaves differently at runtime. For example, `+ +m` (which does nothing) turning into `++m` (which increments `m`).

The easiest and most practical use is branching: the original script runs function `A`, the formatted one runs function `B`. Used like that though, both functions are sitting in both files. Chain enough of them together and you can create bytecode unique to the undiverged/diverged build, where you're essentially using the JS processor itself to decode one program and encode the other, without shortcuts like `eval` / `new Function`. The only way to see the diverged program is to run it through the exact JS processor it was designed to exploit.

How the formatter works

Chromium DevTools uses one of the safest techniques for formatting, a **whitespace-only formatter**. It never actually changes the JavaScript, it just goes through every token and adds whitespace to the original code based on a ruleset.

It also doesn't format by default, only when the file looks minified. `isMinified()` in `front_end/core/text_utils/TextUtils.ts` decides that as an average of 80 or more characters per line (`(length - lineCount) / lineCount >= 80`), so one very long line (just a comment works) forces it on. You can turn it off, but only for that time, clicking the same file again turns it back on.

It's one of the easiest & safest ways to build a formatter, and understanding it makes the more complex tools a lot easier to follow. Take this `input.js`:

```
(()=>{
console.log("Hello World!");
});
```

The formatter parses it with Acorn into an AST and a list of tokens. For every token it looks up the innermost node that directly contains it, and that node's rule returns a short code: `t` writes the token, `s` asks for a space, `n` asks for a newline and `>` indents deeper.

```
=>       ArrowFunctionExpression  ->  sts   space, token, space
{        BlockStatement           ->  tn>   token, newline, indent
console  Identifier               ->  t     token only
```

As it walks the tokens, it's already appending to the output:

```
output = ''
for each token:
    output += (spacing/newline/indent the rules decide)
    output += originalText.substring(token.start, token.end)
```

Which ends up as:

```
( () => {
    console.log("Hello World!");
});
```

Every token is copied straight from the original text, so there's no AST->JS printer involved at all, and the output is almost guaranteed to be the exact same code it was given (excluding the spacing of course). That's why with this type of formatter you'll basically never get anything more than a rerouted divergence.

Stacked unary plus mutates the operand

There's no handler for unary expressions, so two `+` signs separated by a space get glued together.

Original code:

```
var m = 0;
+ +m;
console.log(m);
```

Formatted code:

```
var m = 0;
++m;
console.log(m);
```

`JavaScriptFormatter.ts` has no handler for `UnaryExpression`, so `+` falls through to the default one:

```
return AT.keyword(token) && !AT.keyword(token, 'this') ? 'ts' : 't';
```

Keywords get the token plus a space (`ts`), anything else just gets the token as is (`t`). `+` is a punctuator, not a keyword, so it gets `t`.

There is `FormattedContentBuilder.ts`, which was built to add spaces in cases like `typeof x` and `a in b`, but it never considers two punctuators next to each other, so it doesn't add one either.

So to recap: no handler, default "just write the token" rule, no space from the builder, and the two `+` tokens end up written back to back. `+ +m` (does nothing) becomes `++m` (increments `m`).

Rewritten: WebCrack

The logic is all still there, it's just been rewritten into a different construct that looks like it does the exact same thing, but at runtime it doesn't. For example, `getConfig().ready || (getConfig().ready = true)` (which calls `getConfig` twice) turning into `getConfig().ready ||= true` (which calls it once).

How the transforms work

WebCrack is a Babel based deobfuscator & unminifier. It parses with `@babel/parser`, runs a long list of transforms over the AST and prints it back out with `@babel/generator`. Every transform is tagged as either `safe` or `unsafe`.

Each transform is a small Babel visitor that looks for a specific pattern (using `@codemod/matchers`) and swaps it for a cleaner version. For `a || (a = 1)`, it describes the shape it wants piece by piece and captures the parts it needs:

```
logicalExpression(          a || (a = 1)
  operator,                 ||
  left,                     a          <- captured
  assignmentExpression(     (a = 1)
    '=',
    fromCapture(left),      a          <- has to look the same as the captured one
    right                   1
  )
)
```

If the AST matches, the whole node gets replaced with one built from the captured parts, `a ||= 1`. Pretty much all of WebCrack's "make it readable again" work is done like this, and the matcher only ever checks what the code looks like, never what it actually does when it runs.

Logical assignment evaluates the left side once instead of twice

If the left side has a side effect (e.g. a call), the rewrite only runs it once.

Original code:

```
var calls = 0;
function getConfig() {
  calls++;
  return {};
}
getConfig().ready || (getConfig().ready = true);
console.log(calls);
```

Deobfuscated code:

```
var calls = 0;
function getConfig() {
  calls++;
  return {};
}
getConfig().ready ||= true;
console.log(calls);
```

The transform lives in `src/transpile/transforms/logical-assignments.ts`, and it's tagged `safe`:

```
const left = m.capture(m.or(m.identifier(), m.memberExpression()));
const idMatcher = m.logicalExpression(
  operator,
  left,
  m.assignmentExpression('=', m.fromCapture(left), right),
);

path.replaceWith(
  t.assignmentExpression(operator.current + '=', left.current, right.current),
);
```

`left` accepts any member expression, and `fromCapture(left)` only checks that the second `getConfig().ready` is structurally the same as the first one. Two separate calls to `getConfig()` look exactly the same in the AST, so they match.

But `a.b || (a.b = c)` and `a.b ||= c` are only the same thing when evaluating `a` does nothing at all. The original evaluates `a` twice, `||=` evaluates it once.

kuizuo's js-deobfuscator is a fork of WebCrack and ships the exact same transform, so it diverges the exact same way.

Folded: Restringer

The tool evaluates an expression ahead of time and replaces it with the value it thinks it has, but that guess is wrong for this specific script. For example, `atob('R29vZGJ5ZSwgV29ybGQh')` turning into `'Goodbye, World!'` even though the script replaced `atob` with its own function.

How Restringer works

Restringer is a deobfuscator built on flast, which parses with espree, links every identifier to the place it was declared (`declNode`) and prints with escodegen. Its modules are split into `safe` ones (pattern rewrites) and `unsafe` ones (which actually evaluate code in a sandbox). By default it runs both: the safe ones until nothing changes, then every unsafe one once, and it repeats that until the script stops changing.

Builtin calls get resolved even after the builtin was replaced

Restringer assumes `atob` is always the real `atob`.

Original code:

```
globalThis.atob = function () { return 'Hello, World!'; };
console.log(atob('R29vZGJ5ZSwgV29ybGQh'));
```

Deobfuscated code:

```
globalThis.atob = function () {
  return 'Hello, World!';
};
console.log('Goodbye, World!');
```

`src/modules/unsafe/resolveBuiltinCalls.js` only skips a call if the function being called has a declaration:

```
if (n.callee?.declNode || n?.callee?.object?.declNode || ...) {
  continue;
}

if (n.type === 'CallExpression' &&
AVAILABLE_SAFE_IMPLEMENTATIONS.includes(n.callee.name)) {
  matches.push(n);
}
```

`globalThis.atob = ...` is an assignment to a property, not a declaration, so `atob` never gets a `declNode` and still looks like the global builtin. And `atob` happens to have a "safe implementation" in `src/modules/utils/safe-atob.js`:

```
export function atob(val) {
  return Buffer.from(val, 'base64').toString();
}
```

So the call gets replaced with whatever Node's base64 decode returns for that string, and the patched function never even gets looked at. The best part is the decoy string is sitting right there in the output, looking like the most normal thing in the world.

WebCrack does the exact same thing in `src/deobfuscate/evaluate-globals.ts`, its `path.scope.hasBinding(name, { noGlobals: true })` check also only looks at declared bindings.

Stripped: new-datadome-deobfuscator

The function is still there, but part of what's inside it is gone. Usually it's a call or a `new` that the tool decided doesn't do anything. For example, `return (new Tick(), x)` turning into `return x`.

How new-datadome-deobfuscator works

new-datadome-deobfuscator is a Babel based deobfuscator built specifically for DataDome's scripts (captcha, interstitial and tags). After its preprocessing it runs its transformation passes (t-matrix replacement, conditionals, expressions, if statements, opaque predicates) in a loop until nothing changes, up to 10 times.

`new` is dropped from a sequence

A constructor call inside a comma expression is treated as if it has no side effects.

Original code:

```
var count = 0;
function Tick() {
  count++;
}
function step(x) {
  return (new Tick(), x);
}
step(1);
console.log(count);
```

Deobfuscated code:

```
var count = 0;
function Tick() {
  count++;
}
function step(x) {
  return x;
}
console.log(count);
```

The `SequenceExpression` visitor in `transformers/transformations/expressions.js` goes through every expression in the sequence and only keeps the ones it thinks have side effects:

```
if (isIIFE(expr)) {
  hasSideEffects = true;
}
else if (t.isCallExpression(expr)) {
  hasSideEffects = true;
}

else if (hasSideEffectsOrRuntimeDeps(expr)) {
  hasSideEffects = true;
}
```

In Babel, `new Tick()` is a `NewExpression`, not a `CallExpression`, so it misses every check in that list. The last fallback, `hasSideEffectsOrRuntimeDeps` (`transformers/preprocessing/index.js`), only looks for a `MemberExpression`, a `CallExpression` or a `typeof` anywhere inside, and `new Tick()` contains just the identifier `Tick`. So it's marked "useless", and the sequence gets replaced with the only expression left:

```
if (usefulExpressions.length === 1) {
  path.replaceWith(usefulExpressions[0]);
}
```

On top of that it also removes the `step(1);` call, so the counter is never touched. The exact same thing happens with a tagged template (`tag`x`) and with `delete` in a sequence, since neither is a CallExpression `or a` typeof`.

Pruned: obfuscator-io-deobfuscator

One branch gets judged as dead and deleted, the other one is kept, and there's no trace in the output that the deleted one ever existed. For example, `if (null) { decoy } else {`

`real }` turning into just `decoy`.

How obfuscator-io-deobfuscator works

obfuscator-io-deobfuscator is a Babel based deobfuscator made specifically for javascript-obfuscator (obfuscator.io) output. It parses with `@babel/parser`, runs its transformations (string revealing, control flow unflattening, dead branch removal, etc.) until nothing changes anymore and prints with `@babel/generator`.

`null` is a truthy condition

Any literal that isn't a boolean, number or string is treated as `true`.

Original code:

```
if (null) {
  console.log('decoy');
} else {
  console.log('real');
}
```

Deobfuscated code:

```
console.log('decoy');
```

The `deadBranchRemover` transformation (`deobfuscator/transformations/controlFlow/`) does this for every `if`:

```
IfStatement(path) {
    if (self.isSemiLiteral(path.node.test)) {
        if (self.isTruthy(path.node.test)) {
            const statements = t.isBlockStatement(path.node.consequent)
                ? path.node.consequent.body
                : [path.node.consequent];
            path.replaceWithMultiple(statements);

isSemiLiteral(node) {
    return t.isLiteral(node) || t.isArrayExpression(node) || t.isObjectExpression(node);
}

isTruthy(literal) {
    return t.isBooleanLiteral(literal) || t.isNumericLiteral(literal) ||
t.isStringLiteral(literal)
        ? !!literal.value
        : true;
}
```

In Babel, `t.isLiteral` also matches `NullLiteral`, `BigIntLiteral`, `RegExpLiteral` and `TemplateLiteral`. `isTruthy` only actually checks the value for booleans, numbers and strings, everything else falls through to `true`. That's right for arrays, objects and regexes, but `null`, `0n` and an empty template literal are all falsy.

So `null` counts as a literal, `isTruthy` returns `true` for it, and the whole `if` gets replaced with its consequent, deleting the `else` branch completely. The same happens with `if (0n)`, and with `if (${s})` when `s` is an empty string.

Erased: Terser

The whole function body (or the whole function) is gone. The function still gets called, it just doesn't do anything anymore. For example, `function guard() { var unused = 0 in 1; }` turning into `function guard(){}`.

How Terser works

Terser is the minifier webpack uses by default. It was forked from UglifyJS, parses into its own AST, runs its compress passes over it and prints it back out minified. During compress, every node can answer `has_side_effects()` and `drop_side_effect_free()`, and anything with no side effects whose value isn't used gets dropped.

An `in` that throws is treated as pure

`0 in 1` throws a `TypeError`, but Terser thinks it can't do anything.

Original code:

```
function guard() {
  var unused = 0 in 1;
}
try {
  guard();
  console.log('decoy');
} catch (e) {
  console.log('real');
}
```

Minified code:

```
function guard(){}try{guard(),console.log("decoy")}catch(o){console.log("real")}
```

In `lib/compress/inference.js`, a binary expression only has side effects if one of its sides does, no matter what the operator is:

```
def_has_side_effects(AST_Constant, return_false);
def_has_side_effects(AST_Binary, function(compressor) {
    return this.left.has_side_effects(compressor)
        || this.right.has_side_effects(compressor);
});
```

And `lib/compress/drop-side-effect-free.js` follows the same logic when deciding what can be thrown away:

```
def_drop_side_effect_free(AST_Constant, return_null);
def_drop_side_effect_free(AST_Binary, function (compressor, first_in_statement) {
    var right = this.right.drop_side_effect_free(compressor);
    if (!right)
        return this.left.drop_side_effect_free(compressor, first_in_statement);
```

`0` and `1` are both constants, so both sides return `null` and the whole `0 in 1` is dropped. But `in` with a primitive on the right side throws a `TypeError` at runtime. After that `unused` has nothing left in it and is never read, so the declaration goes too, leaving an empty body. `guard()` no longer throws, so the `try` runs the decoy instead of falling into the `catch`.

UglifyJS (which Terser was forked from) keeps the `0 in 1` here, so this one is Terser's own. swc's minifier, obfuscator-io-deobfuscator and deobfuscate-js erase the same function, and `1n + 1` (which throws because it mixes a BigInt and a Number) gets erased by Terser, UglifyJS and swc.

CSS

This concept works for CSS just as well.

Chromium example (*add a very long html comment to html to make it automatically prettify*)

```
<style>
 .raw{display:none!important}
.raw{display:none}.cr{display:none}.ff{display:none}
.state{&:not(button:hover){.raw{display:block}}}
@media (min-width:1px){[data-k] .cr{display:block}}
.state .ff{display:block
</style>
```

Archived from https://trustsig.eu/blog/javascript-divergence/. Rendered offline from the local Markdown copy.